PRAKTIKUM WEB
R. Sandhika Galih Amalga, ST., MT.
TUGAS - PERTEMUAN 2

Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040084
Nama	Nazala qisti
Kelas	E
Link Github Repository	https://github.com/nazalaemin/praktikum-react-modul-2

Bagian 1. Screenshot Semua Kode

src/tugas-3/CatatanKeuangan.tsx (bagian 1/3)

```tsx
import { useState, useEffect } from 'react';
import Kartu from '../Kartu';
import FormTransaksi from './FormTransaksi';
import ItemTransaksi from './ItemTransaksi';
import { formatRupiah } from './format';
import type { Transaksi, Jenis } from './types';

function CatatanKeuangan() {
  const [daftarTransaksi, setDaftarTransaksi] = useState<Transaksi[]>(() => {
    const dataTersimpan = localStorage.getItem('catatan-keuangan');
    return dataTersimpan ? (JSON.parse(dataTersimpan) as Transaksi[]) : [];
  });

  useEffect(() => {
    localStorage.setItem('catatan-keuangan', JSON.stringify(daftarTransaksi));
  }, [daftarTransaksi]);

  const tambahTransaksi = (data: { keterangan: string; nominal: number; jenis: Jenis }) => {
    const transaksiBaru: Transaksi = { id: Date.now(), ...data };
    setDaftarTransaksi([...daftarTransaksi, transaksiBaru]);
  };

  const hapusTransaksi = (id: number) => {
    setDaftarTransaksi(daftarTransaksi.filter((t) => t.id !== id));
  };

  const totalMasuk = daftarTransaksi
    .filter((t) => t.jenis === 'pemasukan')
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const totalKeluar = daftarTransaksi
```

src/tugas-3/CatatanKeuangan.tsx (bagian 2/3)

```
    .filter((t) => t.jenis === 'pengeluaran')
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const saldo = totalMasuk - totalKeluar;

  return (
    <div>

      <Kartu judul="Saldo Saat Ini" warna={saldo >= 0 ? 'mint' : 'peach'}>
        <p className="saldo">
          {saldo < 0 ? '- ' : ''}{formatRupiah(Math.abs(saldo))}
        </p>
        <div className="chip-baris">
          <span className="chip chip-masuk">Pemasukan {formatRupiah(totalMasuk)}</span>
          <span className="chip chip-keluar">Pengeluaran {formatRupiah(totalKeluar)}</span>
        </div>
      </Kartu>

      <Kartu judul="Tambah Transaksi" warna="sky">
        <FormTransaksi onTambah={tambahTransaksi} />
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        {daftarTransaksi.length === 0 ? (
          <p className="kosong">Belum ada transaksi. Yuk catat transaksi pertamamu!</p>
        ) : (
          <ul className="daftar">
            {daftarTransaksi.map((t) => (
              <ItemTransaksi key={t.id} transaksi={t} onHapus={hapusTransaksi} />
            ))}
          </ul>
```

src/tugas-3/CatatanKeuangan.tsx (bagian 3/3)

```
        )}
      </Kartu>
    </div>
  );
}

export default CatatanKeuangan;
```

src/tugas-3/FormTransaksi.tsx (bagian 1/3)

```tsx
import { useState, type FormEvent } from 'react';
import type { Jenis } from './types';

type DataBaru = { keterangan: string; nominal: number; jenis: Jenis };

type FormTransaksiProps = {
  onTambah: (data: DataBaru) => void;
};

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  const [keterangan, setKeterangan] = useState('');
  const [nominal, setNominal] = useState('');
  const [jenis, setJenis] = useState<Jenis>('pemasukan');

  const nominalAngka = Number(nominal);
  const nominalSalah = nominal !== '' && (isNaN(nominalAngka) || nominalAngka <= 0);
  const tidakValid = keterangan.trim() === '' || nominal === '' || nominalSalah;

  function tanganiSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    if (tidakValid) return;
    onTambah({ keterangan: keterangan.trim(), nominal: nominalAngka, jenis });
    setKeterangan('');
    setNominal('');
    setJenis('pemasukan');
  }

  return (
    <form onSubmit={tanganiSubmit}>
      <div className="kolom">
```

src/tugas-3/FormTransaksi.tsx (bagian 2/3)

```tsx
        <input
          className="input"
          type="text"
          placeholder="Keterangan (contoh: Gaji bulanan)"
          value={keterangan}
          onChange={(e) => setKeterangan(e.target.value)}
        />
        <div className="dua-kolom">
          <input
            className="input"
            type="number"
            placeholder="Nominal (Rp)"
            value={nominal}
            onChange={(e) => setNominal(e.target.value)}
          />
          <select
            className="input"
            value={jenis}
            onChange={(e) => setJenis(e.target.value as Jenis)}
          >
            <option value="pemasukan">Pemasukan</option>
            <option value="pengeluaran">Pengeluaran</option>
          </select>
        </div>
        <button className="tombol" type="submit" disabled={tidakValid}>
          Tambah Transaksi
        </button>
      </div>
      {nominalSalah && <p className="galat">Nominal harus berupa angka lebih dari 0.</p>}
    </form>
```

src/tugas-3/FormTransaksi.tsx (bagian 3/3)

```tsx
  );
}

export default FormTransaksi;
```

src/tugas-3/ItemTransaksi.tsx

```tsx
import type { Transaksi } from './types';
import { formatRupiah } from './format';

type ItemTransaksiProps = {
  transaksi: Transaksi;
  onHapus: (id: number) => void;
};

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
  const pemasukan = transaksi.jenis === 'pemasukan';

  return (
    <li className="item">
      <span>{transaksi.keterangan}</span>
      <strong className={pemasukan ? 'chip chip-masuk' : 'chip chip-keluar'}>
        {pemasukan ? '+' : '-'} {formatRupiah(transaksi.nominal)}
      </strong>
      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;
```

src/Kartu.tsx

```tsx
import type { ReactNode } from 'react';

type Warna = 'putih' | 'butter' | 'sky' | 'mint' | 'peach';

type KartuProps = {
  judul: string;
  warna?: Warna;
  children: ReactNode;
};

function Kartu({ judul, warna = 'putih', children }: KartuProps) {
  return (
    <section className={`kartu kartu-${warna}`}>
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

src/tugas-3/types.ts

```ts
export type Jenis = 'pemasukan' | 'pengeluaran';

export type Transaksi = {
  id: number;
  keterangan: string;
  nominal: number;
  jenis: Jenis;
};
```

src/tugas-3/format.ts

```ts
export function formatRupiah(angka: number): string {
  return 'Rp ' + angka.toLocaleString('id-ID');
}
```

src/App.css (bagian 1/3)

```css
.wadah { max-width: 640px; margin: 0 auto; padding: 48px 20px 64px; }

.judul { font-family: var(--font-judul); font-weight: 600; font-size: 30px; line-height: 1.2; margin: 0 0
.subjudul { color: var(--ink-soft); margin: 0 0 22px; }

/* Kartu: warna menandakan peran isi */
.kartu { background: var(--putih); border: 2px solid var(--ink); border-radius: 18px; padding: 20px 22px
.kartu h2 { font-family: var(--font-judul); font-weight: 500; font-size: 20px; margin: 0 0 14px; }
.kartu-butter { background: var(--butter); }
.kartu-sky { background: var(--sky); }
.kartu-mint { background: var(--mint); }
.kartu-peach { background: var(--peach); }

.baris { display: flex; gap: 8px; flex-wrap: wrap; }
.kolom { display: flex; flex-direction: column; gap: 10px; }
.dua-kolom { display: grid; grid-template-columns: 1fr  1fr ; gap: 10px; }
@media (max-width: 480px) { .dua-kolom { grid-template-columns: 1fr ; } }

/* Input */
.input { flex: 1; min-width: 0; padding: 10px 12px; border: 2px solid var(--ink); border-radius: 12px; ba
.input::placeholder { color: #8E91AA; }
select.input { appearance: none; padding-right: 34px; cursor: pointer; background-image: url("data:image/
.label-tersembunyi { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0);

/* Tombol */
.tombol { padding: 9px 16px; border: 2px solid var(--ink); border-radius: 12px; background: var(--violet
.tombol:hover { background: #B8AEEF; }
.tombol:active { transform: translate(2px, 2px); box-shadow: 0 0 0 var(--ink); }
.tombol:disabled { background: #E6E5EE; color: #8E91AA; border-color: #B9BBCE; box-shadow: none; cursor:
.tombol-merah { background: var(--pink); }
```

src/App.css (bagian 2/3)

```
.tombol-merah:hover { background: #F4BCCD; }
.tombol-kecil { padding: 5px 11px; font-size: 13px; }
.tombol-filter { background: var(--putih); }
.tombol-filter:hover { background: #F1EEFB; }
.tombol-filter.aktif { background: var(--violet); }

:focus-visible { outline: 3px solid var(--fokus); outline-offset: 2px; }

/* Penghitung */
.angka-klik { margin: 0 0 10px; font-size: 16px; }
.pesan-rajin { display: inline-block; margin: 0 0 12px; padding: 4px 12px; background: var(--putih); bor

/* Daftar */
.ringkasan { margin: 0 0 12px; color: var(--ink-soft); font-size: 14px; }
.filter { margin-bottom: 8px; }
.daftar { list-style: none; padding: 0; margin: 0; }
.item { display: flex; align-items: center; gap: 12px; padding: 11px 0; border-bottom: 2px dashed #D9D5E
.item:last-child { border-bottom: none; }
.item span { flex: 1; overflow-wrap: anywhere  ; }
.selesai { text-decoration: line-through; color: #9A9DB5; }
.daftar + .tombol { margin-top: 14px; }

/* Kotak centang */
.item input[type="checkbox"] { appearance: none; flex: none; width: 22px; height: 22px; margin: 0; borde
.item input[type="checkbox"]:checked { background-color: var(--mint); background-image: url("data:image/s

.kosong { margin: 6px 0 0; padding: 20px; border: 2px dashed #C9C5E0; border-radius: 14px; color: var(--
.galat { display: inline-block; margin: 10px 0 0; padding: 4px 12px; background: var(--pink); border-radi

/* Tugas 3 */
```

src/App.css (bagian 3/3)

```
.saldo { font-family: var(--font-judul); font-weight: 600; font-size: 38px; line-height: 1.1; margin: 0 0
.chip-baris { display: flex; gap: 8px; flex-wrap: wrap; }
.chip { display: inline-block; padding: 3px 12px; border-radius: 999px; border: 2px solid var(--ink); fo
.chip-masuk { background: var(--mint); color: #17613F; }
.chip-keluar { background: var(--pink); color: #8A2446; }
.kartu-mint .chip-masuk { background: var(--putih); }
.kartu-peach .chip-keluar { background: var(--putih); }
.kartu-mint .chip-keluar, .kartu-peach .chip-masuk { background: var(--putih); }

/* Tab pemilih tugas */
.tab { display: flex; gap: 8px; margin-bottom: 26px; }

@media (prefers-reduced-motion: reduce) { .tombol { transition: none; } }
```

src/index.css

```css
:root {
  --ink: #2E3150;
  --ink-soft: #666A88;
  --latar: #F4F1FB;
  --violet: #C6BDF2;
  --sky: #CFE5F8;
  --mint: #C9EBDC;
  --peach: #FFD9C6;
  --pink: #F9D2DE;
  --butter: #FFF0B8;
  --putih: #FFFFFF;
  --fokus: #6E8FD6;
  --font-judul: 'Fredoka', 'Trebuchet MS', system-ui, sans-serif;
  --font-isi: 'DM Sans', system-ui, -apple-system, 'Segoe UI', sans-serif;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  background-color: var(--latar);
  background-image: radial-gradient(#DDD7F2 1.2px, transparent 1.2px);
  background-size: 22px 22px;
  color: var(--ink);
  font-family: var(--font-isi);
  font-size: 15px;
  line-height: 1.5;
}
```

Bagian 2. Penjelasan Kode

Aplikasi dibagi menjadi tiga komponen dan dua berkas pendukung. CatatanKeuangan menjadi komponen induk yang memegang seluruh data. FormTransaksi dan ItemTransaksi adalah komponen anak, sedangkan Kartu adalah komponen pembungkus yang dipakai ulang. Alasannya, data hanya boleh disimpan di satu tempat (induk), sehingga tampilan di semua anak selalu konsisten.

1. types.ts: definisi tipe data

Tipe Jenis dibatasi hanya 'pemasukan' atau 'pengeluaran', dan tipe Transaksi berisi id, keterangan, nominal (number), dan jenis. Dengan begitu TypeScript langsung menandai kesalahan, misalnya jika nominal tidak sengaja diisi teks atau jenis salah ketik.

2. format.ts: fungsi formatRupiah

Fungsi ini mengubah angka menjadi tulisan seperti "Rp 4.850.000" memakai toLocaleString('id-ID'). Fungsi dipisah ke berkas sendiri karena dipakai di dua tempat (ringkasan saldo dan ItemTransaksi), sehingga cukup ditulis satu kali.

3. Kartu.tsx: komponen pembungkus

Kartu menerima props judul, warna, dan children. Props children berisi apa pun yang diletakkan di antara <Kartu> dan </Kartu>, sehingga satu komponen bisa membungkus isi yang berbeda-beda (saldo, formulir, daftar). Props warna dipakai untuk memilih className, misalnya kartu-mint atau kartu-sky.

4. CatatanKeuangan.tsx: komponen induk

State daftarTransaksi dibuat dengan useState berisi fungsi, bukan nilai langsung. Fungsi tersebut membaca localStorage dengan kunci "catatan-keuangan" lalu mengubahnya kembali menjadi array memakai JSON.parse.

Cara ini hanya dijalankan sekali saat aplikasi pertama tampil, sehingga localStorage tidak dibaca berulang kali setiap render.

Hook useEffect dengan dependency array [daftarTransaksi] menyimpan data ke localStorage lewat JSON.stringify setiap kali daftar berubah. Penyimpanan ke localStorage adalah efek samping (sinkronisasi ke luar React), jadi tidak boleh ditulis langsung di badan komponen. localStorage hanya bisa menyimpan teks, karena itu array harus diubah menjadi JSON terlebih dahulu.

Fungsi tambahTransaksi memakai setDaftarTransaksi([...daftarTransaksi, transaksiBaru]). Spread operator membuat array baru, bukan mengubah array lama dengan push. React mendeteksi perubahan dengan membandingkan data lama dan baru, sehingga jika array lama diubah langsung (mutasi), React menganggap tidak ada yang berubah dan layar tidak diperbarui. Fungsi hapusTransaksi memakai .filter() yang juga menghasilkan array baru tanpa transaksi yang id-nya sama.

Nilai totalMasuk, totalKeluar, dan saldo dihitung dari state memakai .filter() dan .reduce() pada setiap render. Ketiganya sengaja tidak disimpan sebagai state baru, karena nilai yang bisa dihitung dari state lain jika disimpan terpisah berisiko tidak sinkron dengan data aslinya.

Pada JSX, warna kartu saldo memakai ternary (mint jika saldo >= 0, peach jika negatif), dan pesan "Belum ada transaksi" juga memakai ternary berdasarkan panjang array. Daftar dirender dengan .map() dan atribut key={t.id} agar React dapat mengenali setiap item. Fungsi tambahTransaksi dan hapusTransaksi dikirim ke anak lewat props (onTambah dan onHapus); pola ini disebut lifting state up: anak tidak mengubah data sendiri, melainkan "melapor" ke induk.

5. FormTransaksi.tsx: formulir terkendali

Formulir memiliki tiga state lokal (keterangan, nominal, jenis). Setiap input dipasangi value dan onChange sehingga isi input selalu sama dengan state (controlled form). Dengan cara ini data input bisa langsung dipakai untuk validasi.

Nominal disimpan sebagai string karena nilai dari input selalu berupa teks. Sebelum divalidasi dan dikirim, nilainya diubah dengan Number(nominal). Langkah ini penting agar penjumlahan menjadi perhitungan angka; tanpa konversi, "5000" + "2000" menghasilkan "50002000".

Variabel nominalSalah dan tidakValid memeriksa keterangan tidak kosong dan nominal berupa angka lebih dari 0. Hasilnya dipakai untuk menonaktifkan tombol (disabled={tidakValid}) dan menampilkan pesan galat dengan operator && (conditional rendering). Fungsi tanganiSubmit diawali event.preventDefault() agar halaman tidak reload saat formulir dikirim. Setelah itu fungsi onTambah dipanggil untuk melapor ke induk dan semua input dikosongkan kembali. Pada onSubmit={tanganiSubmit} fungsi ditulis tanpa tanda kurung, artinya fungsi diberikan ke React untuk dijalankan nanti, bukan dipanggil saat render.

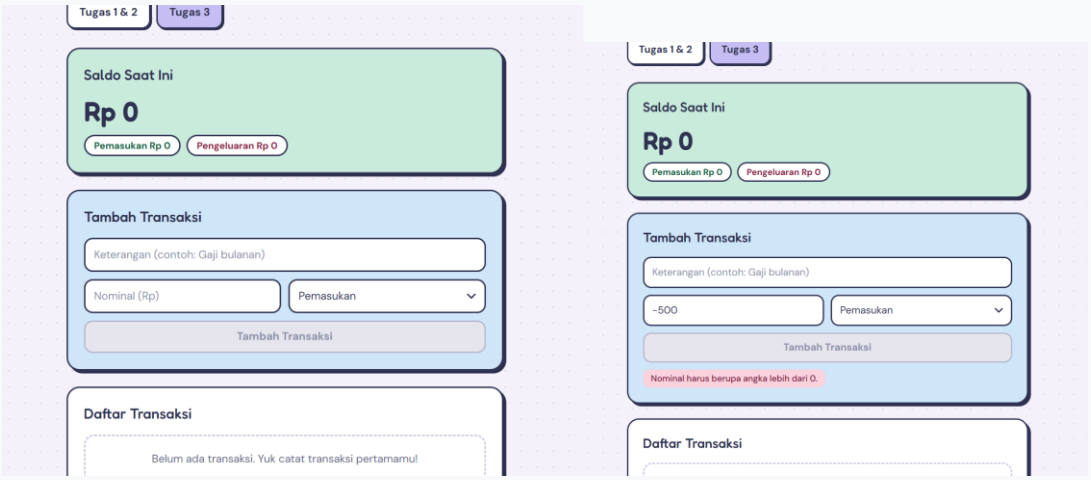
6. ItemTransaksi.tsx: satu baris transaksi

Komponen menerima satu objek transaksi dan fungsi onHapus. Nominal diberi className dinamis lewat ternary: chip-masuk (hijau) untuk pemasukan dan chip-keluar (merah) untuk pengeluaran, lengkap dengan tanda + atau - . Tombol hapus memakai onClick={() => onHapus(transaksi.id)}. Arrow function diperlukan karena fungsi perlu membawa id; jika ditulis onClick={onHapus(transaksi.id)}, fungsi langsung berjalan saat render dan transaksi terhapus sendiri.

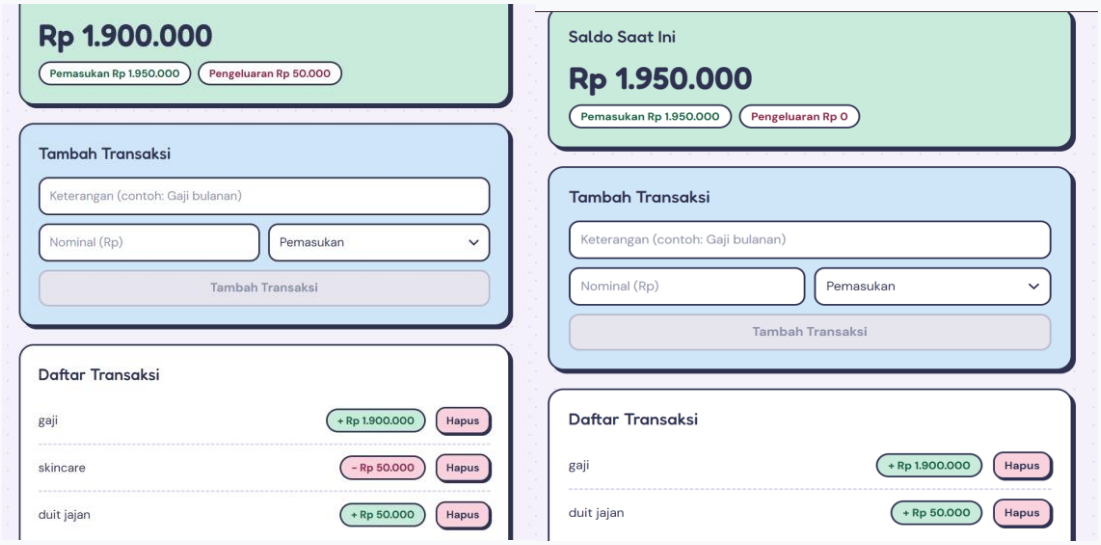
7. App.css dan index.css: gaya tampilan

index.css berisi variabel warna pastel pada :root (misalnya --mint, --pink, --sky), pengaturan font, dan latar halaman. App.css memakai variabel tersebut untuk kartu, tombol, kolom formulir, dan label nominal. Warna ditulis sebagai variabel agar bisa diganti dari satu tempat.

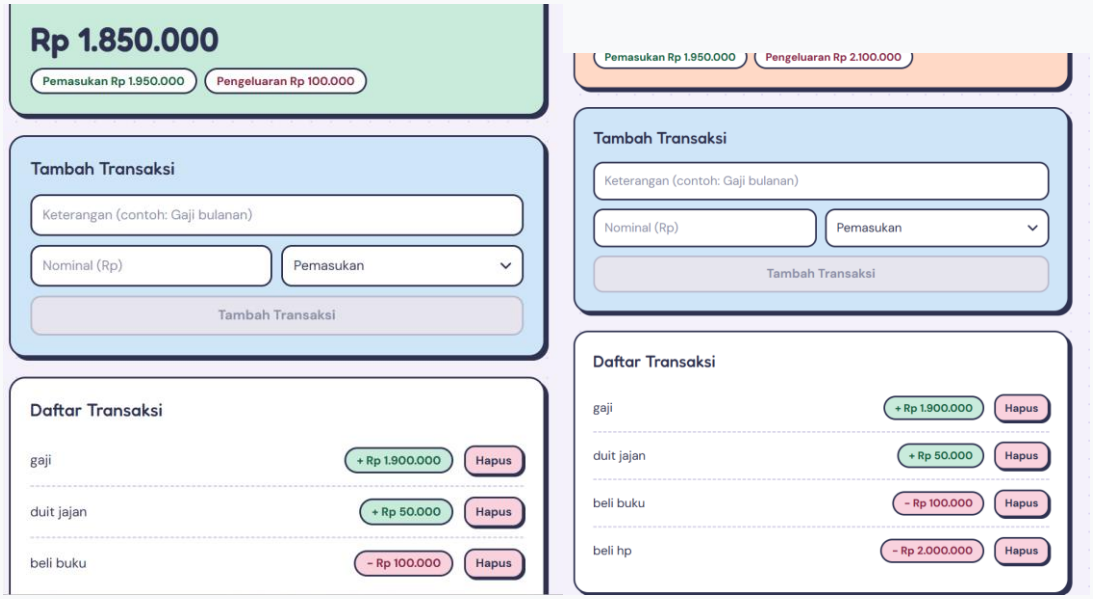
Bagian 3. Screenshot Hasil Antarmuka



Gambar 3.1 Kondisi awal (kiri) dan validasi nominal salah (kanan)



Gambar 3.2 Setelah empat transaksi ditambahkan (kiri) dan setelah satu transaksi dihapus (kanan)



Gambar 3.3 Setelah halaman di-refresh (kiri) dan saat saldo negatif (kanan)

Bagian 4. Penjelasan Hasil Antarmuka

Gambar 3.1 (kiri): kondisi awal

Saat belum ada data, saldo bernilai Rp 0, kedua label pemasukan dan pengeluaran bernilai Rp 0, dan daftar menampilkan pesan "Belum ada transaksi". Pesan ini muncul karena panjang array daftarTransaksi sama dengan 0 (conditional rendering dengan ternary). Tombol "Tambah Transaksi" nonaktif karena formulir masih kosong, sehingga tidak ada transaksi kosong yang bisa tersimpan.

Gambar 3.1 (kanan): validasi

Ketika nominal diisi -500, pesan galat "Nominal harus berupa angka lebih dari 0" muncul dan tombol tetap nonaktif. Ini bisa terjadi karena input terkendali: isi input tersimpan di state, sehingga variabel nominalSalah dihitung ulang pada setiap ketikan dan hasilnya langsung memengaruhi tampilan.

Gambar 3.2 (kiri): menambah transaksi

Empat transaksi ditambahkan: dua pemasukan (Rp 1.500.000 dan Rp 400.000) dan dua pengeluaran (Rp 35.000 dan Rp 120.000). Pemasukan tampil hijau dan pengeluaran tampil merah. Saldo menjadi Rp 1.745.000, yaitu 1.900.000 dikurangi 155.000. Angka saldo ikut berubah otomatis karena dihitung dari state, bukan ditulis manual. Selain itu, nominal dijumlahkan sebagai angka karena sudah diubah dengan Number() sebelum disimpan.

Gambar 3.2 (kanan): menghapus transaksi

Transaksi "Makan siang di kantin" dihapus lewat tombol Hapus. Item hilang dari daftar, dan saldo naik menjadi Rp 1.780.000 karena pengeluaran Rp 35.000 tidak lagi dihitung. Hal ini terjadi karena hapusTransaksi menghasilkan array baru dengan .filter(), sehingga React merender ulang daftar dan ringkasan.

Gambar 3.3 (kiri): data bertahan setelah refresh

Setelah halaman di-refresh (F5), daftar transaksi dan saldo tetap sama. Padahal state React hanya hidup di memori dan hilang saat halaman dimuat ulang. Data tetap ada karena useEffect menyimpannya ke localStorage setiap kali daftar berubah, lalu useState membacanya kembali saat aplikasi pertama tampil.

Gambar 3.3 (kanan): saldo negatif

Ketika pengeluaran lebih besar dari pemasukan (misalnya menambah "Bayar kos" Rp 2.000.000), saldo tampil dengan tanda minus dan kartu saldo berubah menjadi warna peach. Perubahan warna ini berasal dari ternary pada props warna di komponen Kartu, yang bergantung pada nilai saldo. Dengan demikian pengguna langsung mengetahui bahwa keuangannya defisit.